

**Autonomous Precision Agriculture Robot for Real-Time Soil
Moisture Mapping and
Crop Health Monitoring
(Mid cycle Report)**

Karthik Ganta

Master's Project in Cyber-physical systems

Advisor: Dr Rolando Herrero

Date: March,30,2023

1. Abstract

Precision agriculture requires intelligent and automated systems to optimize resource utilization and improve crop productivity. This project presents the design and implementation of an Autonomous Precision Agriculture Robot built on an embedded Linux platform, capable of real-time sensing, processing, and remote control. The system integrates an MPU6050 Inertial Measurement Unit (IMU) for motion and orientation sensing, along with a multi-threaded software architecture to ensure efficient and non-blocking operation.

A dedicated sensor thread continuously acquires and processes accelerometer and gyroscope data, while a TCP-based communication module enables real-time remote monitoring and control of the robot. The system supports command-based actuation, allowing users to control movement and interact with the robot over a network. The modular design ensures scalability for future integration of computer vision, autonomous navigation, and IoT-based cloud connectivity.

Experimental results demonstrate stable sensor acquisition, reliable communication, and effective system integration. The proposed system provides a strong foundation for developing intelligent, autonomous agricultural solutions aimed at reducing manual labor, optimizing water usage, and improving overall farming efficiency.

2. Objectives

The primary objectives of this project are:

- Detect dry and wet land regions
- Enable real-time soil and environmental monitoring
- Provide remote control capabilities
- Develop a scalable embedded system architecture
- Reduce manual labor and improve efficiency

3. System Architecture

3.1 Hardware Components

Raspberry pi: It is the main brain of the entire system which runs all the algorithm

DC Motor: Helps the robot to navigate.

Servo Motor: This will help the robot to change the angle navigate right or left

Motor Driver: uses I2C to driver the servo motor and motor driver.

3.2 Software Design and Implementation

IMU Sensor Integration

The MPU6050 sensor is interfaced using the I2C protocol via the wiringPi library. Key configuration registers such as PWR_MGMT_1, SMPLRT_DIV, and GYRO_CONFIG are initialized to enable stable operation.

Raw sensor data is read and converted into physical units:

- Acceleration (g)
- Angular velocity (°/s)

Data Processing

Sensor readings are processed using scaling factors:

- Accelerometer: raw / 16384.0
- Gyroscope: raw / 131.0

Reusable functions were implemented:

- read_accel_data()
- read_gyro_data()

Multithreading Architecture

A dedicated thread is implemented using POSIX threads (pthreads) for continuous sensor data acquisition.

Features:

- Real-time IMU data reading
- Separation of sensor logic from control logic

Communication System (TCP Server)

A **multi-threaded TCP server** is implemented to enable remote monitoring and control.

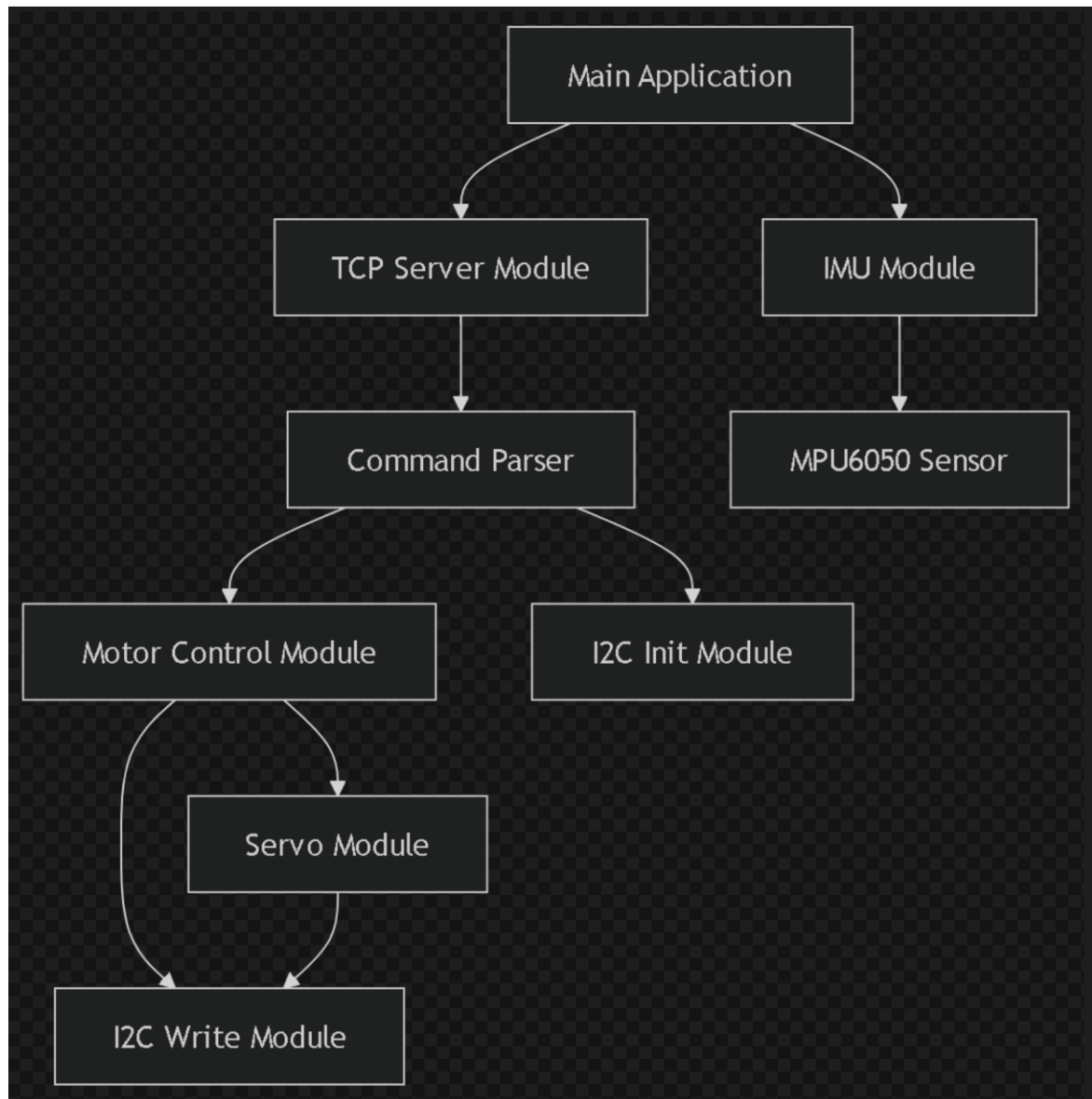
Key Features

- Listens for incoming client connections
- Handles multiple clients using threads
- Receives control commands (e.g., Forward, Backward, Left, Right, Stop)
- Parses commands and triggers motor control actions

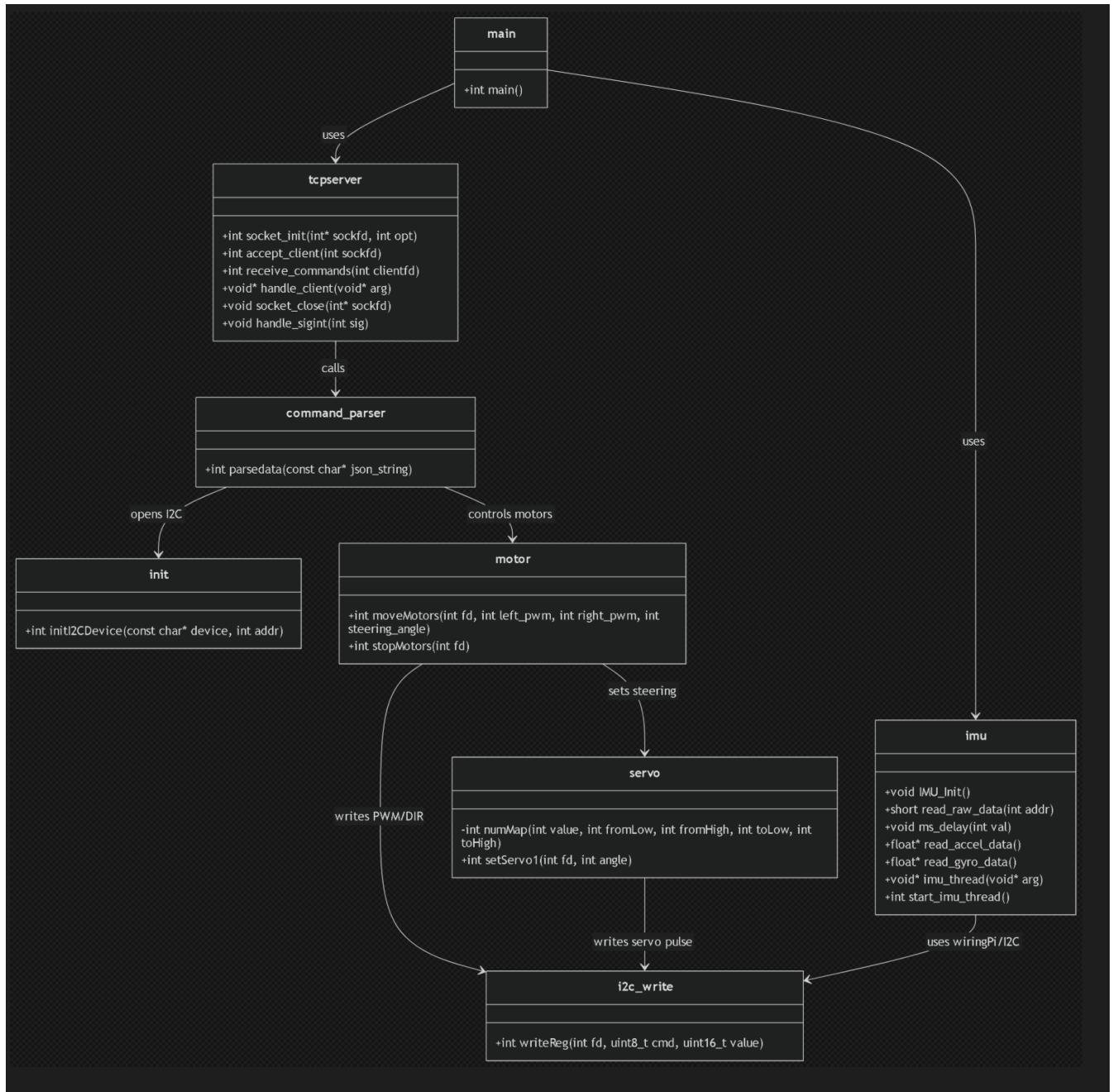
System Workflow

1. Client sends command over TCP
2. Server receives and parses input
3. Command is mapped to motor control logic
4. Robot executes the action

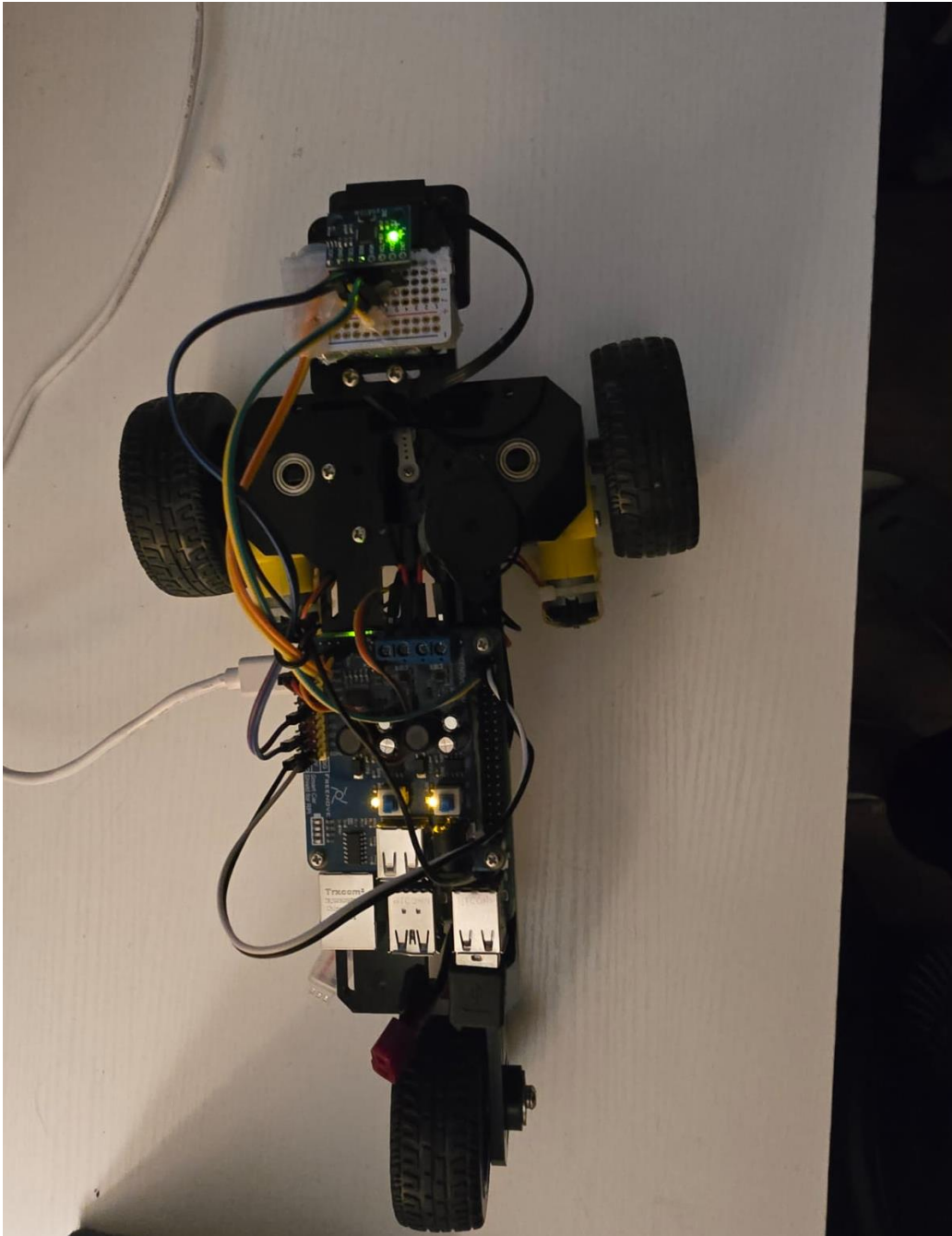
4 Code Bock Diagram



5. Complete UML Diagram

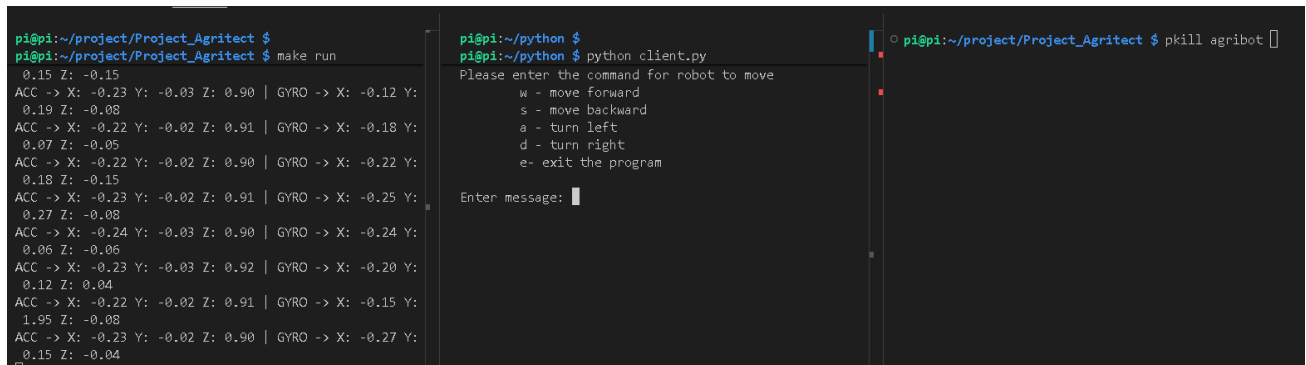


6. System Diagram



7. Result and observation

On the left screen shows the live values of the accelerometer and gyro and on the right side shows the command to move the robot



```
pi@pi:~/project/Project_Agritect $
pi@pi:~/project/Project_Agritect $ make run
0.15 Z: -0.15
ACC -> X: -0.23 Y: -0.03 Z: 0.90 | GYRO -> X: -0.12 Y:
0.19 Z: -0.08
ACC -> X: -0.22 Y: -0.02 Z: 0.91 | GYRO -> X: -0.18 Y:
0.07 Z: -0.05
ACC -> X: -0.22 Y: -0.02 Z: 0.90 | GYRO -> X: -0.22 Y:
0.18 Z: -0.15
ACC -> X: -0.23 Y: -0.02 Z: 0.91 | GYRO -> X: -0.25 Y:
0.27 Z: -0.08
ACC -> X: -0.24 Y: -0.03 Z: 0.90 | GYRO -> X: -0.24 Y:
0.06 Z: -0.06
ACC -> X: -0.23 Y: -0.03 Z: 0.92 | GYRO -> X: -0.20 Y:
0.12 Z: 0.04
ACC -> X: -0.22 Y: -0.02 Z: 0.91 | GYRO -> X: -0.15 Y:
1.95 Z: -0.08
ACC -> X: -0.23 Y: -0.02 Z: 0.90 | GYRO -> X: -0.27 Y:
0.15 Z: -0.04
□

pi@pi:~/python $
pi@pi:~/python $ python client.py
Please enter the command for robot to move
w - move forward
s - move backward
a - turn left
d - turn right
e- exit the program

Enter message:

○ pi@pi:~/project/Project_Agritect $ kill agribot □
```

8. Future Enhancements

Navigation System

- Path planning and obstacle avoidance
- Sensor fusion for localization

Computer Vision

- Soil moisture detection using image processing
- Crop disease detection using machine learning

Data Fusion

- Implementation of complementary/Kalman filters
- Improved orientation estimation

9. Conclusion

This project successfully demonstrates the development of an embedded, multi-threaded robotic system for precision agriculture. The integration of sensor data acquisition, real-time processing, and network communication establishes a strong foundation for autonomous farming solutions.

The system showcases practical applications of embedded Linux, system programming, and IoT communication, making it highly scalable for future enhancements such as computer vision and autonomous navigation.

GitHub Link : https://github.com/Ganta-Karthik1999/Project_Agritect